



Université Abdelmalek Essadi

Faculté des Sciences et Techniques de Tanger

Master IASD M1 S2

Module : NLP – Natural Language Processing

Snort RAG Rule Generator

Conception et validation d'un système NLP/RAG pour la
génération contrôlée de règles Snort 3

Réalisé par :

KHALOUFI Youssef

LEHOUEIMEL Yahfdhou Mohammed

ALLOUCH Oussama

EL KAIRAH Hassan

LOUAHABI Abdenour

Encadrante : Pr. Ikram BENABDELOUAHAB

Année universitaire : 2025–2026

13 juin 2026

Remerciements

Nous remercions l'encadrante du projet pour les orientations pédagogiques autour de la détection d'intrusion, de Snort et des architectures NLP/RAG. Nous remercions également les membres de l'équipe pour la constitution du corpus, les scripts de validation et la consolidation des preuves expérimentales.

Résumé

Ce rapport présente un système académique défensif qui transforme une description d'attaque ou un journal réseau en une règle Snort 3 candidate. Le pipeline combine détection du type d'attaque, récupération d'exemples contrôlés, génération structurée, validation locale, réparation limitée, solution de repli déterministe, analyse heuristique du risque de faux positifs et validation expérimentale séparée avec Snort 3 et fichiers PCAP. La révision complète du rapport s'appuie sur des preuves recalculées dans `report/evidence/`. Le corpus officiel contient 200 lignes, dont 180 exemples malveillants et 20 exemples bénins. L'export officiel contient 180 SID issus du dataset ; le fichier Snort 3 validé en contient 183, car trois règles supplémentaires sont réservées aux scénarios PCAP contrôlés. Les tests automatisés courants donnent 31/31 tests réussis en 53,35 s. Les résultats doivent être interprétés comme une validation académique contrôlée, non comme une qualification opérationnelle en production.

Mots-clés : Snort 3, IDS, NLP, RAG, BM25, TF-IDF, validation syntaxique, rejeu PCAP, faux positifs.

Abstract

This report studies a defensive academic system that transforms an attack description or a network log into a candidate Snort 3 rule. The pipeline combines attack-type detection, controlled example retrieval, structured generation, local validation, limited repair, deterministic fallback, heuristic false-positive analysis, and separate experimental validation with Snort 3 and PCAP files. The revised report relies on recomputed evidence stored under `report/evidence/`. The official corpus contains 200 rows, including 180 malicious examples and 20 benign examples. The official dataset export contains 180 SIDs; the Snort 3 rule file contains 183 because three additional rules are dedicated to controlled PCAP scenarios. The current automated test run reports 31/31 tests passing in 53.35 s. Results are academic and controlled, not production certification claims.

Keywords : Snort 3, IDS, NLP, RAG, BM25, TF-IDF, syntax validation, PCAP replay, false positives.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Table des matières	vi
Table des figures	vii
Liste des tableaux	viii
Liste des acronymes	ix
1 Introduction générale	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Questions de recherche	1
1.4 Contributions	2
1.5 Organisation du rapport	2
2 État de l’art, travaux connexes et concepts techniques	3
2.1 Détection par signatures	3
2.2 Récupération d’information	3
2.3 Génération augmentée par récupération	3
2.4 Génération structurée contrainte	3
2.5 Comparaison synthétique	4
2.6 Synthèse	4
3 Méthodologie	5
3.1 Source de vérité	5
3.2 Protocoles expérimentaux	5
3.3 Diagramme A : pipeline de requête en ligne	6
3.4 Diagramme B : validation expérimentale hors ligne	7
3.5 Diagramme C : exécution Docker/Snort	8
3.6 Critères d’interprétation	8

4	Corpus, provenance et audit des données	9
4.1	Corpus officiel	9
4.2	Schéma et exemples	9
4.3	Audit des doublons et distributions	10
4.4	Provenance externe	11
4.5	Limites de sécurité	11
5	Architecture et génération contrôlée	12
5.1	Vue d'ensemble	12
5.2	Noms d'architectures	12
5.3	Schéma JSON strict	13
5.4	Génération et repli	13
5.5	Règle annotée	13
6	Validation et sécurité	15
6.1	Quatre couches distinctes	15
6.2	Validation locale	15
6.3	Validation Snort 3	15
6.4	Validation PCAP	16
6.5	Analyse heuristique des faux positifs	17
7	Résultats expérimentaux	18
7.1	RQ1–RQ2 : récupération	18
7.2	RQ3–RQ4 : génération LLM contrôlée	19
7.3	RQ5 : validation des règles	19
7.4	RQ6–RQ7 : rejeu PCAP	20
7.5	Clustering	20
7.6	Comparaison des architectures	21
7.7	Tests automatisés	22
8	Menaces à la validité et améliorations	23
8.1	Validité interne	23
8.2	Validité de construit	23
8.3	Validité externe	23
8.4	Validité de conclusion	23
8.5	Reproductibilité	23
8.6	Améliorations prioritaires	24
9	Organisation du projet et contributions	25
9.1	Organisation technique	25
9.2	Contributions par membre	25

10 Démonstration complète de la plateforme	26
10.1 Onglet Rule Generator	27
10.2 Onglet Retrieval Details	28
10.3 Onglet Validation & False Positives	29
10.4 Onglet Dataset / Knowledge Base	30
10.5 Onglet Benchmarks & Evidence	31
10.6 Onglet PDF Upload	32
10.7 Démonstration bénigne contrôlée	33
10.8 Synthèse de bout en bout	34
11 Conclusion générale	35
Références bibliographiques	36

Table des figures

3.1	Pipeline en ligne exposé par le tableau de bord et l'API.	6
3.2	Chaîne de validation expérimentale hors ligne.	7
3.3	Organisation de l'exécution Snort 3 via le wrapper Docker.	8
4.1	Distribution des catégories du corpus officiel.	11
6.1	Succès binaire des comportements attendus lors du rejeu PCAP.	16
7.1	Précision@3 de la récupération avec et sans champs révélateurs.	18
7.2	Modes de génération observés dans les résultats LLM sauvegardés.	19
7.3	Métriques de clustering recalculées sans champs d'étiquette directs.	20
7.4	Projection t-SNE du clustering sans étiquette.	21
7.5	Métriques contrôlées des variantes d'architecture.	21
10.1	Onglet de génération de règle	27
10.2	Détails de récupération	28
10.3	Validation locale et faux positifs	29
10.4	Corpus et base de connaissances	30
10.5	Benchmarks et preuves sauvegardées	31
10.6	Extension temporaire par PDF	32
10.7	Décision bénigne contrôlée	33

Liste des tableaux

2.1	Travaux connexes et positionnement du projet.	4
4.1	Catégories d'attaque du corpus officiel.	9
4.2	Exemple compact d'une ligne du dataset.	10
4.3	Exemple manuel et variation contrôlée.	10
4.4	Séparation des sources de données.	11
5.1	Responsabilités des principaux modules.	12
5.2	Champs attendus de la sortie LLM contrôlée.	13
5.3	Exemples de comportements de génération.	13
6.1	Couches de validation et portée des preuves.	15
6.2	Synthèse des scénarios PCAP recalculés.	17
7.1	Ablation de récupération à k=3 sur dix requêtes contrôlées.	18
7.2	Synthèse de la campagne de tests automatisés.	22
9.1	Contributions techniques synthétiques par membre.	25
10.1	Synthèse fonctionnelle de la démonstration Gradio.	34

Liste des acronymes

API	interface de programmation d'application
ARI	indice de Rand ajusté
BM25	Best Matching 25
CSV	fichier à valeurs séparées par des virgules
FAISS	Facebook AI Similarity Search
HTTP	Hypertext Transfer Protocol
ICMP	Internet Control Message Protocol
IDS	système de détection d'intrusion
JSON	JavaScript Object Notation
LLM	grand modèle de langue
MRR	rang réciproque moyen
NMI	information mutuelle normalisée
NLP	traitement automatique du langage naturel
PCAP	capture de paquets réseau
RAG	génération augmentée par récupération
SID	identifiant de signature Snort
SVD	décomposition en valeurs singulières
TCP	Transmission Control Protocol
TF-IDF	pondération terme-fréquence – inverse document-fréquence
UDP	User Datagram Protocol

1 Introduction générale

1.1 Contexte

Les systèmes de détection d'intrusion par signatures restent utiles lorsque les comportements recherchés sont connus et décrits par des indicateurs précis. Snort 3 fournit un moteur de règles largement documenté, mais l'écriture manuelle de signatures exige une connaissance simultanée du protocole, du format des journaux, de la syntaxe Snort et du risque de faux positifs [1]. Le projet étudié cherche à assister cette écriture dans un cadre académique contrôlé.

Dans ce rapport, la récupération désigne la recherche d'exemples pertinents dans un corpus local. Le reclassement désigne la réorganisation d'un Top-k initial par heuristiques supplémentaires. L'ancrage documentaire désigne ici la validité des identifiants de sources utilisés par une sortie structurée ; il ne mesure pas une justification sémantique complète. La solution de repli déterministe désigne le générateur local utilisé lorsque la sortie LLM est absente ou invalide. Le tableau de bord expose ces fonctions à l'utilisateur, tandis que le rejeu de fichiers PCAP appartient à la validation expérimentale hors ligne.

1.2 Problématique

La problématique est la suivante : dans quelle mesure un système NLP/RAG contrôlé peut-il produire des règles Snort 3 candidates, traçables et validables, à partir de descriptions ou de journaux réseau, sans présenter comme automatiques des garanties qui relèvent en réalité de validations expérimentales séparées ?

1.3 Questions de recherche

Le rapport est organisé autour de huit questions :

- **RQ1.** Avec quelle précision la récupération identifie-t-elle des exemples contrôlés pertinents ?
- **RQ2.** Quelle est la contribution de BM25, de TF-IDF, de la fusion hybride et du reclassement ?
- **RQ3.** À quelle fréquence les LLM locaux produisent-ils directement des sorties contraintes valides ?
- **RQ4.** À quelle fréquence la réparation ou la solution de repli déterministe est-elle nécessaire ?

- **RQ5.** Les règles exportées passent-elles la validation locale et la validation à l'exécution avec Snort 3 ?
- **RQ6.** Les scénarios PCAP contrôlés déclenchent-ils les familles de règles attendues ?
- **RQ7.** L'ensemble bénin contrôlé reste-t-il silencieux ?
- **RQ8.** Quelles sont les principales menaces à la validité ?

1.4 Contributions

Les contributions techniques vérifiables sont les suivantes :

- un corpus officiel de 200 lignes équilibrées sur dix catégories ;
- une couche de récupération lexicale creuse, BM25, hybride et reclassement ;
- une génération contrôlée avec validation JSON, réparation limitée et solution de repli déterministe ;
- une validation locale des règles, une validation Snort 3, un jeu PCAP contrôlé et une analyse heuristique des faux positifs ;
- une révision expérimentale sauvegardée dans `report/evidence/`.

1.5 Organisation du rapport

Le chapitre 2 présente l'état de l'art et les concepts techniques. Le chapitre 3 décrit la méthodologie et les protocoles expérimentaux. Le chapitre 4 analyse le corpus. Le chapitre 5 décrit l'architecture réelle. Le chapitre 6 sépare les couches de validation. Le chapitre 7 répond aux questions de recherche à partir des artefacts recalculés. Le chapitre 8 discute les limites et menaces à la validité. Le chapitre 9 décrit les contributions de manière factuelle. Le chapitre 10 conclut en répondant à la problématique.

2 État de l’art, travaux connexes et concepts techniques

2.1 Détection par signatures

Snort 3 représente les comportements détectables sous forme de règles composées d’un en-tête et d’options telles que `msg`, `content`, `flow`, `classtype`, `sid` et `rev` [1]. Cette approche est explicable et vérifiable, mais elle dépend de la qualité des signatures et peut produire des faux positifs lorsque les conditions sont trop larges.

2.2 Récupération d’information

TF-IDF pondère les termes par fréquence locale et rareté globale [2]. BM25 prolonge ce principe par une normalisation de longueur et une saturation de fréquence [3]. Dans ce projet, TF-IDF et BM25 sont des méthodes de récupération lexicale creuse, non des représentations denses. Sentence-BERT [4] et FAISS [5] existent dans le code comme backends optionnels, mais leur disponibilité est vérifiée au moment de l’exécution.

2.3 Génération augmentée par récupération

Le RAG combine récupération et génération afin de fournir un contexte documentaire au modèle génératif [6]. Dans un contexte de sécurité, cette approche ne suffit pas à prouver qu’une règle est correcte : il faut vérifier le format de sortie, les identifiants de sources, la syntaxe Snort et le comportement sur trafic contrôlé.

2.4 Génération structurée contrainte

Le projet impose une sortie JSON puis vérifie les champs attendus. Cette contrainte réduit les erreurs de format, mais les résultats recalculés montrent que les LLM locaux sauvegardés s’appuient fortement sur la solution de repli déterministe. Il faut donc distinguer le succès du pipeline final et le succès brut du LLM.

2.5 Comparaison synthétique

TABLEAU 2.1 : Travaux connexes et positionnement du projet.

Domaine	Technique	Forces	Limites et différence avec ce projet
IDS signatures	par Règles Snort 3 [1]	Syntaxe explicable et moteur validable	Ne génère pas automatiquement les règles; notre projet ajoute NLP/RAG et validation locale.
Recherche lexicale	TF-IDF [2], BM25 [3]	Simple, rapide, interprétable	Sensible aux termes révélateurs; le rapport ajoute une ablation sans étiquette.
Recherche sémantique	Sentence-BERT [4], FAISS [5]	Similarité dense et indexation efficace	Backends optionnels; non utilisés comme preuve principale quand indisponibles.
RAG	Récupération puis génération [6]	Contexte documentaire explicite	Ne prouve pas la validité syntaxique ou comportementale; notre pipeline ajoute validation et repli.
Inférence locale	Ollama [7]	Exécution locale de modèles	Les versions, quantifications et ressources ne sont pas toutes disponibles dans les artefacts sauvegardés.
Interfaces et API	Gradio [8], FastAPI [9]	Démonstration et endpoints testables	Le tableau de bord ne lance pas automatiquement Snort ou les PCAP pour chaque requête.

2.6 Synthèse

La contribution du projet n’est pas un nouvel algorithme de détection d’intrusion. Elle réside dans l’intégration contrôlée de récupération, génération structurée, validation syntaxique, validation Snort 3, rejeu PCAP et documentation expérimentale reproductible.

3 Méthodologie

3.1 Source de vérité

La révision considère le dépôt courant comme source de vérité. Les données originales et les anciens résultats ne sont pas écrasés. Les preuves propres au rapport sont recalculées et sauvegardées sous `report/evidence/`. La copie de sauvegarde de la version précédente se trouve dans `report_before_full_revision/`.

3.2 Protocoles expérimentaux

Les tests automatisés sont exécutés avec `PYTHONPATH=src pytest tests/ -ra --durations=10`. Le résultat sauvegardé indique 31 tests réussis en 53,35 s. Le corpus est audité par comptages, doublons, valeurs manquantes et similarité TF-IDF entre descriptions. La récupération est évaluée sur les dix requêtes contrôlées de `src/snort_rag/evaluation.py`.

Deux représentations sont comparées pour la récupération : la représentation complète actuelle et une représentation sans étiquette qui retire `attack_type`, `attack_family` et les règles dont le message révèle la classe. Le clustering est également recalculé sur une représentation sans étiquette, car le module original inclut des champs révélateurs.

3.3 Diagramme A : pipeline de requête en ligne

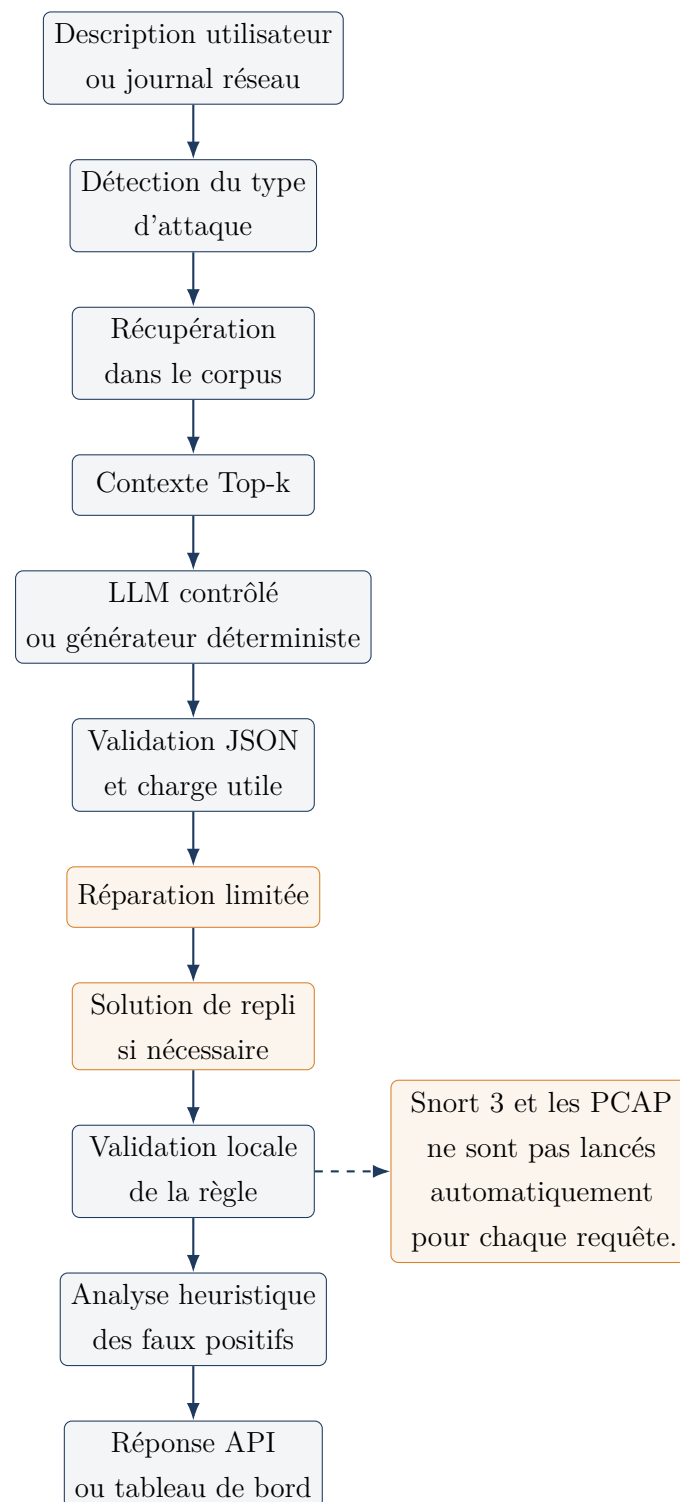


FIGURE 3.1 : Pipeline en ligne exposé par le tableau de bord et l'API.

La validation Snort 3 et le rejeu PCAP ne sont pas exécutés automatiquement pour chaque requête du tableau de bord dans le code audité. Ils appartiennent aux scripts expérimentaux.

3.4 Diagramme B : validation expérimentale hors ligne

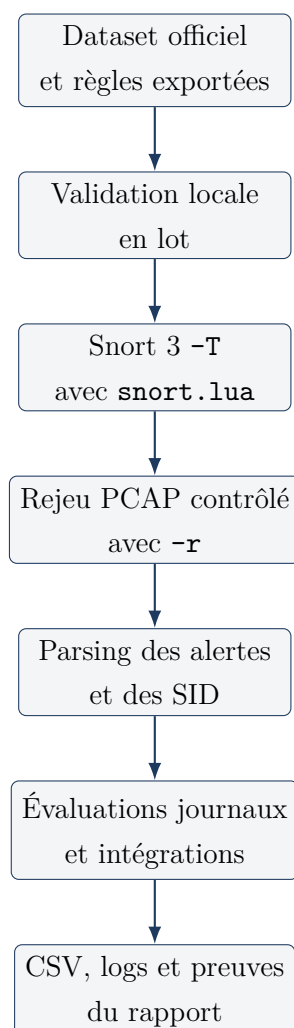


FIGURE 3.2 : Chaîne de validation expérimentale hors ligne.

3.5 Diagramme C : exécution Docker/Snort

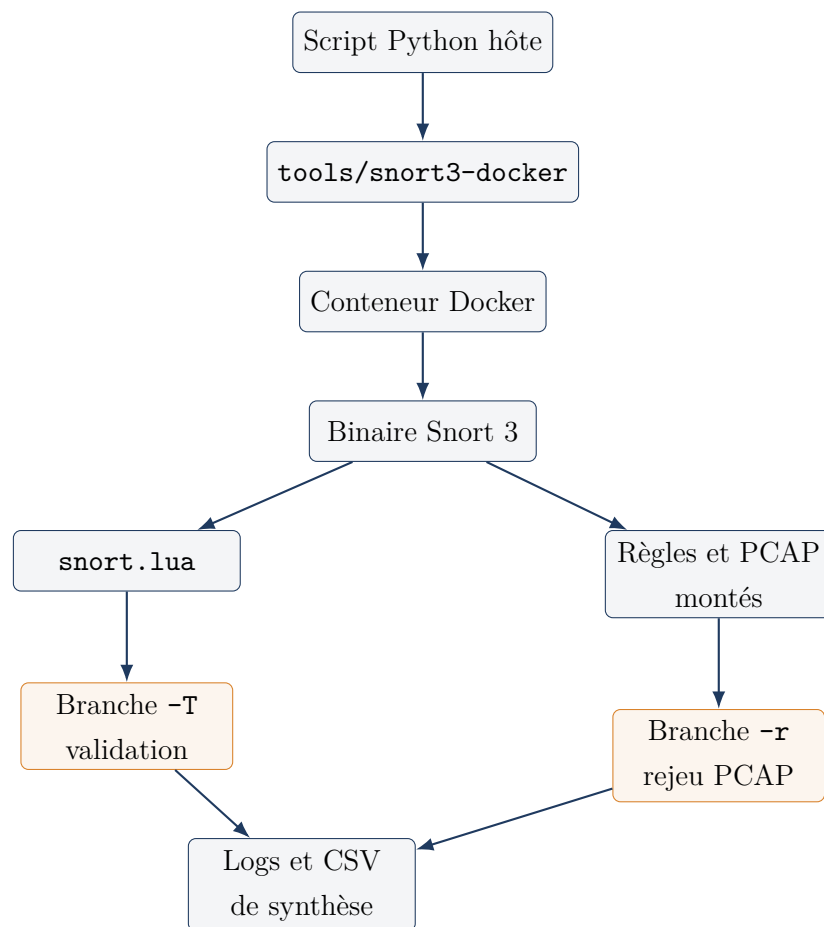


FIGURE 3.3 : Organisation de l'exécution Snort 3 via le wrapper Docker.

3.6 Critères d'interprétation

Les métriques de récupération répondent à RQ1 et RQ2. Les modes de génération répondent à RQ3 et RQ4. Les validations locales et Snort répondent à RQ5. Le rejeu PCAP répond à RQ6 et RQ7. Les menaces à la validité sont discutées explicitement au chapitre 8.

4 Corpus, provenance et audit des données

4.1 Corpus officiel

Le corpus officiel est `data/processed/final_snort_dataset.csv`. L'audit recalculé dans `report/evidence/dataset_audit.json` indique 200 lignes, dix catégories équilibrées de 20 lignes, 180 exemples malveillants et 20 exemples bénins. Les sources internes sont 80 lignes manuelles et 120 variations synthétiques contrôlées. Le corpus n'est pas du trafic de production.

TABLEAU 4.1 : Catégories d'attaque du corpus officiel.

Catégorie	Définition utilisée dans le corpus
<code>port_scan</code>	Tentatives répétées de connexion ou de sondage de ports.
<code>ssh_bruteforce</code>	Échecs SSH répétés ou comportement d'authentification forcée.
<code>sql_injection</code>	Injection SQL dans une URI ou un paramètre HTTP.
<code>xss</code>	Charge utile JavaScript ou HTML interprétable côté client.
<code>command_injection</code>	Paramètre HTTP menant à l'exécution de commandes système.
<code>directory_traversal</code>	Accès à des chemins sensibles par séquences <code>../</code> .
<code>dns_tunneling</code>	Requêtes DNS longues ou encodées associées à un tunnel.
<code>icmp_sweep</code>	Balayage ICMP de plusieurs hôtes internes.
<code>malware_c2</code>	Beacon ou communication de commande et contrôle.
<code>benign_traffic</code>	Trafic normal pour lequel aucune règle n'est recommandée.

4.2 Schéma et exemples

Chaque ligne contient un identifiant, une description naturelle, un type d'attaque, une famille, un protocole, des ports, une sévérité, un journal exemple, une règle Snort de référence ou `NO_RULE_RECOMMENDED`, un contexte de faux positifs, un type de source et une explication attendue.

TABLEAU 4.2 : Exemple compact d’une ligne du dataset.

Champ	Valeur abrégée
Identifiant	SNORT-P1-0001
Catégorie	port_scan
Journal	SYN depuis 198.51.100.10 vers plusieurs ports internes.
Règle	alert tcp ... flags:S; detection_filter:... sid:9100001; rev:1;
Risque bénin	Scanner de vulnérabilité autorisé.

TABLEAU 4.3 : Exemple manuel et variation contrôlée.

Type	Description	Rôle
Manuel	Détection d’un SYN scan externe sur plusieurs ports internes.	Exemple initial révisable.
Variation	Reformulation avec autres ports, horodatage et journal synthétique cohérent.	Augmentation contrôlée, non trafic réel.

4.3 Audit des doublons et distributions

L’audit ne trouve aucun identifiant dupliqué, aucune description strictement dupliquée, aucune ligne strictement dupliquée et aucune valeur manquante. La recherche de quasi-doublons sur les descriptions, avec un seuil de similarité cosinus TF-IDF de 0,85, trouve 4 paires. Ce résultat est cohérent avec un corpus équilibré et partiellement synthétique.

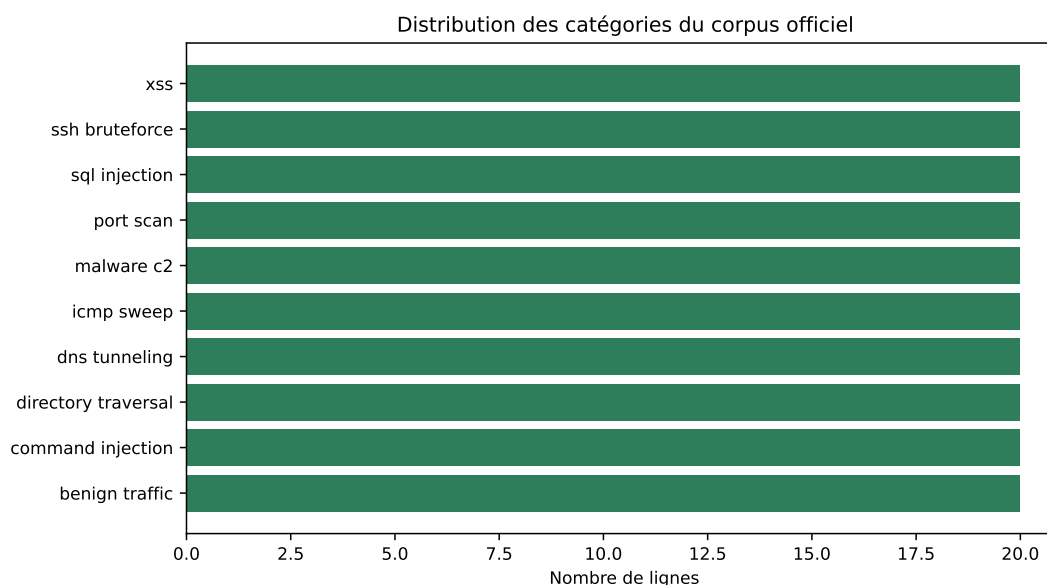


FIGURE 4.1 : Distribution des catégories du corpus officiel.

4.4 Provenance externe

La base de connaissances externe optionnelle est séparée du dataset officiel. Le fichier `data/knowledge_base/fetch_summary.json` recense Emerging Threats Open Rules et Snort Community Rules, téléchargés comme archives de règles ouvertes. Ces sources peuvent enrichir la récupération, mais ne doivent pas être confondues avec le corpus officiel.

TABLEAU 4.4 : Séparation des sources de données.

Source	Usage	Statut
Dataset officiel	Évaluation principale et génération contrôlée	200 lignes
Base externe de règles	Connaissance optionnelle séparée	72 494 lignes extraites
PDF téléversés	Contexte temporaire ajouté par l'utilisateur	Non fiable par défaut

4.5 Limites de sécurité

Les PDF téléversés sont des documents non fiables. Leur texte peut contenir des indications erronées, des règles obsolètes ou des instructions adversariales. Le rapport ne les utilise pas comme preuve principale. Les résultats présentés reposent sur les fichiers sauvegardés du dépôt et les recomputations sous `report/evidence/`.

5 Architecture et génération contrôlée

5.1 Vue d'ensemble

L'application est organisée en modules Python sous `src/snort_rag/`. Les modules principaux sont la récupération, les architectures RAG, le générateur déterministe, le générateur LLM contrôlé, le parseur de règles, l'analyse des faux positifs, le tableau de bord Gradio et l'API FastAPI.

TABLEAU 5.1 : Responsabilités des principaux modules.

Module	Responsabilité
<code>retrieval.py</code>	Index TF-IDF, BM25, fusion hybride, reclassement et import PDF optionnel.
<code>architectures.py</code>	Comparaison des variantes sans RAG, RAG classique, hybride, multi-hop, graphe léger et routage adaptatif.
<code>llm_generator.py</code>	Prompt strict, extraction JSON, validation de charge utile, réparation et repli.
<code>generator.py</code>	Génération déterministe, choix de règle et construction de réponse.
<code>rule_parser.py</code>	Normalisation et validation locale d'un sous-ensemble Snort 2/3.
<code>false_positive.py</code>	Score heuristique et suggestions de réduction des faux positifs.

5.2 Noms d'architectures

Le terme *Agentic RAG* du code correspond à un routage adaptatif inspiré de l'Agentic RAG : la décision est déterministe et dépend de la spécificité de la requête, des mots-clés détectés et d'une éventuelle expansion de requête. Il ne s'agit pas d'un agent autonome.

Le terme *Graph RAG* correspond à une représentation graphe légère organisée par type d'attaque. Le code groupe les documents par `attack_type` ; il ne crée ni base de graphe, ni graphe de connaissances complet.

5.3 Schéma JSON strict

TABLEAU 5.2 : Champs attendus de la sortie LLM contrôlée.

Champ	Contrainte
<code>attack_type</code>	Une valeur de la liste autorisée dans <code>prompting.py</code> .
<code>generated_rule</code>	Règle Snort valide ou <code>NO_RULE_RECOMMENDED</code> pour le bénin.
<code>used_source_doc_ids</code>	Liste d'identifiants appartenant au Top-k récupéré.
<code>explanation</code>	Justification textuelle courte.
<code>false_positive_notes</code>	Note qualitative sur les risques de faux positifs.
<code>confidence</code>	Score numérique fourni par le modèle, non calibré.

Le nombre maximal de réparations est 1 dans `generate_with_llm_context`. Si la sortie reste invalide, le système utilise la solution de repli déterministe. La vérification des identifiants de sources est une validité d'identifiants, non une preuve d'ancrage sémantique.

5.4 Génération et repli

Le générateur déterministe cherche d'abord une règle récupérée de même catégorie et localement valide. Si aucune règle adaptée n'est disponible, il produit un modèle de règle local. La stratégie SID des modèles locaux repose sur les gabarits du projet ; les exports officiels du dataset utilisent les SID 9100001 à 9100180. Les trois SID 9100181–9100183 sont des règles supplémentaires du fichier Snort 3 pour les PCAP.

TABLEAU 5.3 : Exemples de comportements de génération.

Cas	Interprétation
Sortie JSON malformée	Réparation demandée avec les erreurs de validation et les identifiants autorisés.
Réparation réussie	La sortie est acceptée seulement si la règle et les sources sont valides.
Repli déterministe	Utilisé lorsque le LLM échoue ; ce succès ne doit pas être compté comme succès brut LLM.
Trafic bénin	La sémantique attendue est <code>NO_RULE_RECOMMENDED</code> .

5.5 Règle annotée

Listing 5.1 : Règle Snort annotée issue du dataset.

```
1 alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (  
2   msg:"P1 SQL injection payload 1";  
3   flow:to_server,established;  
4   content:"union select",nocase;  
5   classtype:web-application-attack;  
6   sid:9100041;  
7   rev:1;  
8 )
```

Cette règle cible un flux HTTP entrant, exige un contenu caractéristique, déclare une classification, un SID et une révision. Elle reste une signature contrôlée et doit être validée avec Snort avant tout usage.

6 Validation et sécurité

6.1 Quatre couches distinctes

Le rapport distingue quatre couches de validation. Les confondre conduirait à surestimer la portée des résultats.

TABLEAU 6.1 : Couches de validation et portée des preuves.

Couche	Entrée	Outil	Critère	Ce que cela ne prouve pas
Parseur local	Règle texte	<code>rule_parser.py</code>	En-tête et options minimales valides	Chargement réel par Snort ou absence de faux positifs.
Snort 3	Fichier de règles	<code>snort -T</code> via wrapper	Code retour et CSV PASS	Détection comportementale sur trafic.
PCAP contrôlé	Règles et PCAP	<code>snort -r</code>	SID attendu ou silence bénin	Performance comparative générale.
Faux positifs	Règle candidate	Heuristiques locales	Score et facteurs de risque	Mesure empirique sur trafic réel.

6.2 Validation locale

Le fichier `report/evidence/local_rule_validation.csv` contient une ligne par entrée du dataset avec SID, validité locale, erreurs, options détectées, options manquantes et couverture d'options. Les lignes bénignes sont considérées valides lorsqu'elles utilisent `NO_RULE_RECOMMENDED`.

6.3 Validation Snort 3

Le CSV historique `results/snort_runtime_validation.csv` contient 183 lignes en statut **PASS**. La réconciliation recalculée montre que 180 SID proviennent du dataset officiel et que 3 SID supplémentaires appartiennent au fichier Snort 3 utilisé pour les scénarios

PCAP. Il faut donc écrire : 180 règles exportées depuis le dataset, et 183 règles présentes dans le fichier Snort 3 validé.

6.4 Validation PCAP

Le fichier `report/evidence/pcap_detailed_audit.csv` remplace l'interprétation naïve « une alerte suffit ». Pour les scénarios malveillants, le critère exige un SID attendu de la famille contrôlée. Pour le scénario bénin, le critère exige l'absence d'alerte.

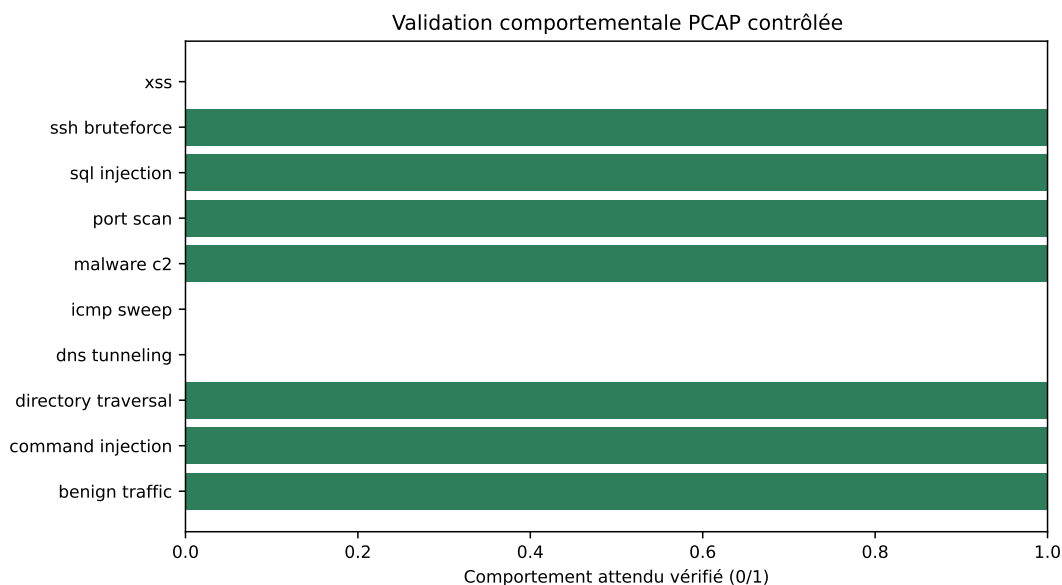


FIGURE 6.1 : Succès binaire des comportements attendus lors du rejeu PCAP.

TABLEAU 6.2 : Synthèse des scénarios PCAP recalculés.

Scénario	Paquets	Alertes	Critère
benign_traffic	5	0	aucune alerte (PASS)
command_injection	5	1	au moins un SID attendu de la famille contrôlée (PASS)
directory_traversal	5	3	au moins un SID attendu de la famille contrôlée (PASS)
dns_tunneling	1	1	au moins un SID attendu de la famille contrôlée (FAIL)
icmp_sweep	15	15	au moins un SID attendu de la famille contrôlée (FAIL)
malware_c2	30	2	au moins un SID attendu de la famille contrôlée (PASS)
port_scan	11	3	au moins un SID attendu de la famille contrôlée (PASS)
sql_injection	5	1	au moins un SID attendu de la famille contrôlée (PASS)
ssh_bruteforce	40	24	au moins un SID attendu de la famille contrôlée (PASS)
xss	5	1	au moins un SID attendu de la famille contrôlée (FAIL)

6.5 Analyse heuristique des faux positifs

Les contrôles négatifs ajoutés dans l'artefact dédié aux faux positifs montrent que les règles trop larges reçoivent un risque supérieur. L'heuristique détecte fortement le cas `any any -> any any`; les autres contrôles reçoivent un risque faible mais non nul. Cette analyse reste une alerte qualitative, pas une mesure empirique.

7 Résultats expérimentaux

7.1 RQ1–RQ2 : récupération

L’ablation sauvegardée dans `report/evidence/retrieval_ablation_summary.csv` compare la représentation complète et la représentation sans étiquette. La représentation complète bénéficie de champs révélateurs tels que `attack_type`, `attack_family` et certaines règles. La représentation sans étiquette est donc plus stricte.

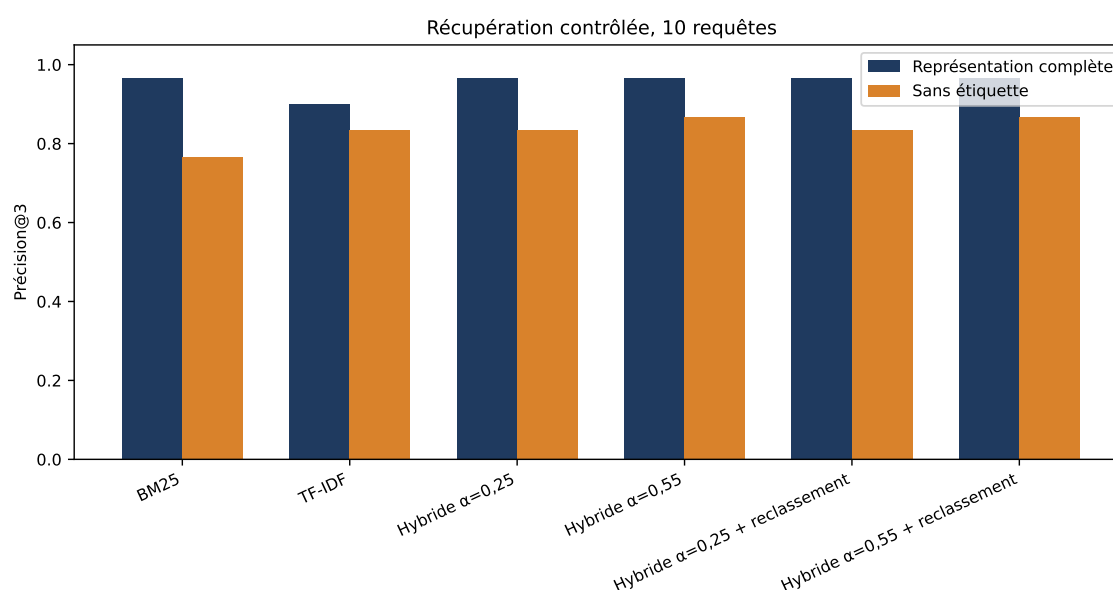


FIGURE 7.1 : Précision@3 de la récupération avec et sans champs révélateurs.

TABLEAU 7.1 : Ablation de récupération à $k=3$ sur dix requêtes contrôlées.

Méthode	Préc. complète	Préc. sans étiquette	Rappel sans étiquette	Latence ms
BM25	0.97	0.77	0.80	7.44
TF-IDF	0.90	0.83	0.90	6.69
Hyb. 0,25	0.97	0.83	0.90	6.72
Hyb. 0,55	0.97	0.87	0.90	6.80
Hyb. 0,25 + R	0.97	0.83	0.90	6.74
Hyb. 0,55 + R	0.97	0.87	0.90	6.84

Interprétation. La représentation complète atteint des scores plus élevés. La représentation sans étiquette reste performante mais révèle une baisse mesurable, ce qui confirme que les anciennes métriques ne devaient pas être interprétées sans audit de fuite.

7.2 RQ3–RQ4 : génération LLM contrôlée

Les résultats LLM sauvegardés couvrent 60 lignes et trois tags Ollama : `qwen2.5`, `llama3.2` et `mistral`. Le résumé recalculé indique 12 sorties directement valides par `rag_llm` et 48 solutions de repli déterministes. Le taux final de validité du pipeline contrôlé est de 1,0, mais il ne correspond pas au succès brut du LLM.

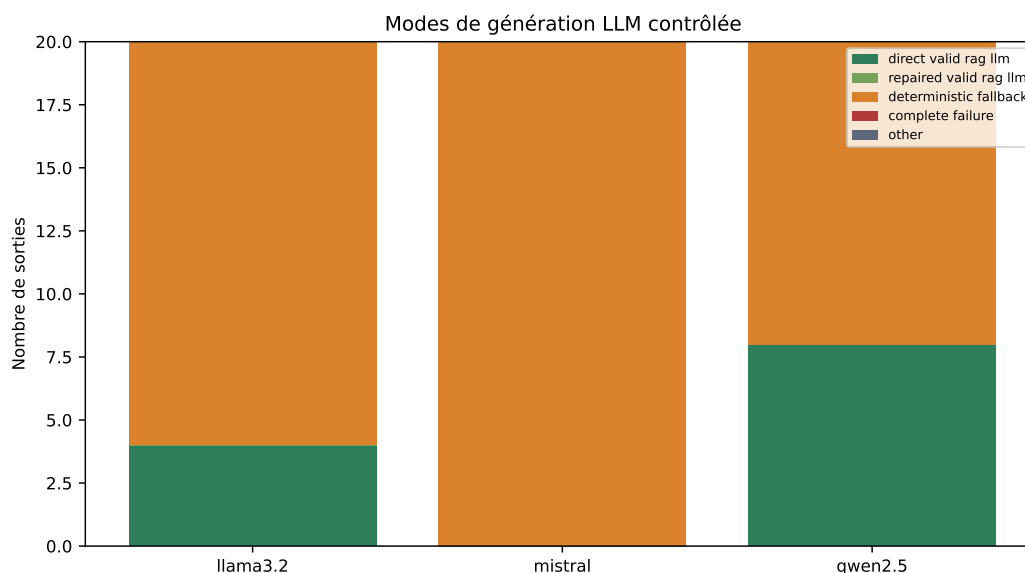


FIGURE 7.2 : Modes de génération observés dans les résultats LLM sauvegardés.

La latence moyenne sauvegardée est de 21 626,36 ms, la médiane de 25 161,57 ms, avec un minimum de 5601,08 ms et un maximum de 51 853,33 ms. Les versions exactes d'Ollama, les quantifications et la politique de cache ne sont pas disponibles dans les artefacts ; le rapport ne les invente donc pas.

7.3 RQ5 : validation des règles

Le dataset officiel contient 180 SID uniques. Les fichiers `person1_rules.rules` et `person1_rules_snort3.rules` ne représentent pas exactement la même surface : le fichier Snort 3 contient trois règles supplémentaires, 9100181, 9100182 et 9100183, utilisées par les scénarios PCAP XSS, DNS tunneling et ICMP sweep. La preuve complète se trouve dans `report/evidence/rules_count_reconciliation.csv`.

7.4 RQ6–RQ7 : rejou PCAP

L’audit détaillé montre que les scénarios `xss`, `dns_tunneling` et `icmp_sweep` déclenchent les trois SID supplémentaires plutôt que les SID du dataset officiel. Ils ne satisfont donc pas le critère strict « SID attendu de la famille dataset », même si le CSV historique les marque `PASS` parce que Snort a traité le PCAP sans erreur. Le scénario bénin reste silencieux.

7.5 Clustering

Le module original de clustering incluait des champs révélateurs. La recomputation sans étiquette est documentée dans `report/evidence/clustering_feature_audit.md`. Les métriques restent très élevées, ce qui s’explique par la structure fortement contrôlée et templatisée du corpus ; elles ne doivent pas être interprétées comme preuve de généralisation.

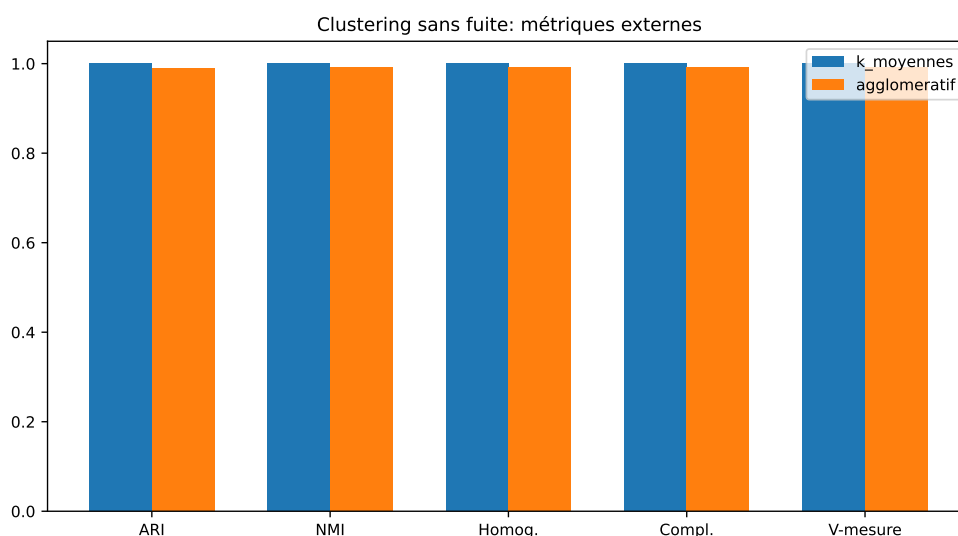


FIGURE 7.3 : Métriques de clustering recalculées sans champs d’étiquette directs.

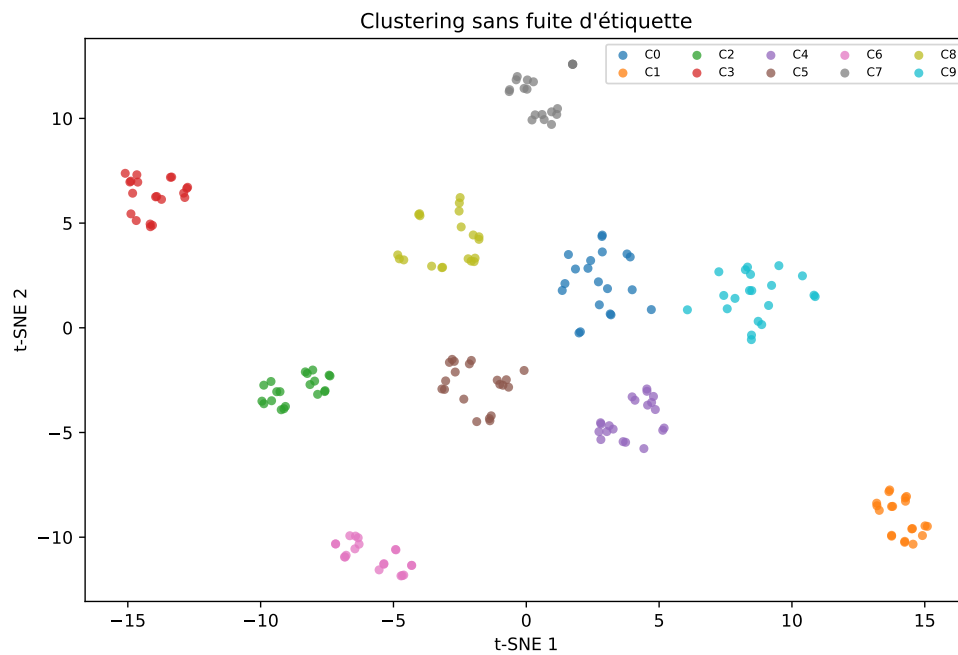


FIGURE 7.4 : Projection t-SNE du clustering sans étiquette.

7.6 Comparaison des architectures

La figure 7.5 remplace le graphique de risque d'hallucination. Le rapport conserve les métriques observables : validité finale, disponibilité d'une preuve de récupération et couverture d'options. Le risque d'hallucination du code reste une heuristique interne, non un taux empirique.

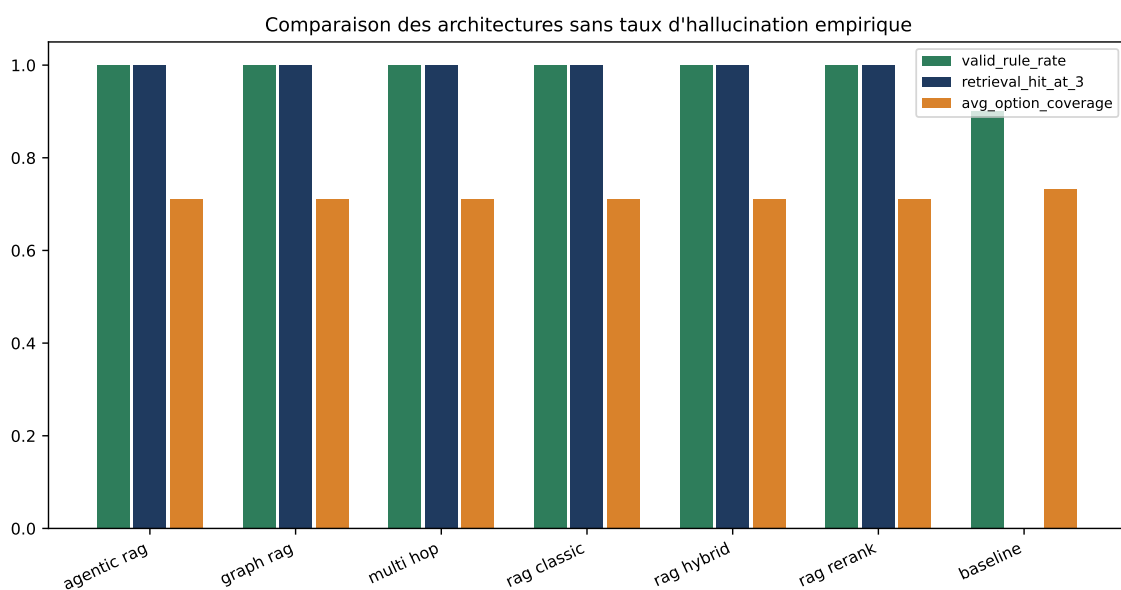


FIGURE 7.5 : Métriques contrôlées des variantes d'architecture.

7.7 Tests automatisés

La commande sauvegardée `PYTHONPATH=src pytest tests/ -ra --durations=10` a collecté 31 tests et les 31 ont réussi en 53,35 s. Le détail brut figure dans `report/evidence/pytest_output.txt`; le résumé structuré est conservé dans `report/evidence/pytest_summary.json`.

TABLEAU 7.2 : Synthèse de la campagne de tests automatisés.

Élément	Valeur vérifiée
Commande	<code>PYTHONPATH=src pytest tests/ -ra --durations=10</code>
Tests collectés	31
Résultat	31 réussites, 0 échec
Durée totale	53,35 s
Test le plus long	Génération du manifeste PCAP dans <code>tests/test_pcap_scripts.py</code> (8,38 s)

Ces tests couvrent l'API, le tableau de bord, le générateur, l'intégration des journaux, la récupération, le parseur de règles et les scripts de validation. Ils ne remplacent pas la validation comportementale avec Snort et PCAP; ils confirment seulement que les chemins automatisés du dépôt restent cohérents après la révision du rapport.

8 Menaces à la validité et améliorations

8.1 Validité interne

Les métriques de récupération et de clustering sont sensibles aux champs utilisés pour construire les vecteurs. Le code original incluait des champs d'étiquette. La révision ajoute des ablations sans étiquette, mais le corpus reste très structuré et partiellement synthétique.

8.2 Validité de construit

La validité des identifiants de sources n'est pas une preuve d'ancrage sémantique complet. Le score de faux positifs est heuristique. Le risque d'hallucination dans les anciens résultats est une approximation interne et ne doit pas être présenté comme un taux empirique.

8.3 Validité externe

Le corpus contient 200 lignes équilibrées, ce qui facilite l'analyse mais ne reflète pas les distributions réelles. Les PCAP sont contrôlés et synthétiques ; un seul scénario bénin est disponible. Les affirmations ne s'étendent donc pas à un réseau de production.

8.4 Validité de conclusion

Les dix requêtes contrôlées limitent la puissance statistique. Les paramètres de fusion et de reclassement sont sélectionnés manuellement. Les résultats LLM dépendent des modèles locaux, de leur configuration et du matériel, dont plusieurs détails ne sont pas disponibles dans les artefacts.

8.5 Reproductibilité

Les scripts et résultats recalculés sont regroupés dans `report/evidence/`. Les métadonnées d'environnement indiquent le commit Git, la date, l'OS, Python, les versions principales de packages, Snort observé, les modèles Ollama cités dans les CSV et les seeds. Les valeurs indisponibles sont explicitement marquées comme telles.

8.6 Améliorations prioritaires

Les améliorations prioritaires sont : augmenter le nombre de requêtes, ajouter des PCAP bénins variés, séparer un jeu de réglage et un jeu d'évaluation, mesurer les faux positifs sur trafic long, documenter précisément les versions et quantifications LLM, et remplacer les heuristiques de risque par des validations comportementales plus étendues.

9 Organisation du projet et contributions

9.1 Organisation technique

Le dépôt sépare le code applicatif, les scripts expérimentaux, les données, les résultats, les tests, la documentation et le rapport. Cette organisation facilite la reproduction partielle des preuves sans modifier les artefacts originaux.

9.2 Contributions par membre

TABLEAU 9.1 : Contributions techniques synthétiques par membre.

Membre	Contribution factuelle
KHALOUFI Youssef : Leader	Architecture initiale, base fonctionnelle et intégration finale du pipeline : récupération, génération contrôlée, faux positifs, Snort 3, PCAP, journaux, benchmark LLM, tests, tableau de bord et preuves.
LEHOUEIMEL Yahfdhou Mohammed	Tableau de bord Gradio et API FastAPI : statistiques, classification, clustering, visualisations, démonstrations, téléversement PDF, validation et faux positifs.
LOUAHABI Abdenour	Benchmark de récupération et embeddings : scripts, métriques Precision@k, Hit@k, Recall@k, MRR, graphiques, résultats et documentation.
ALLOUCH Oussama	Validation locale du dataset et des règles Snort : syntaxe, protocoles, direction, SID, révision, options, <code>.gitignore</code> et hygiène du dépôt.
EL KAIRAH Hassan	Ajustements dans <code>generator.py</code> , résultats comparatifs et première évolution du générateur.

Les contributions sont attribuées à partir des différences techniques observables entre les versions successives du projet. Elles décrivent les principales responsabilités visibles dans le dépôt et ne constituent pas une mesure exacte du temps de travail individuel.

10 Démonstration complète de la plateforme

Cette section présente l'utilisation réelle du tableau de bord Gradio implémenté dans `src/snort_rag/app_gradio.py`. L'interface se lance depuis la racine du dépôt avec :

```
1 PYTHONPATH=src python -m snort_rag.app_gradio
```

Le tableau de bord donne une vue interactive sur les mêmes modules que ceux décrits dans les chapitres précédents : récupération, génération, validation locale, analyse de faux positifs, consultation du corpus, preuves expérimentales sauvegardées et extension temporaire par PDF. La démonstration principale utilise la requête « Detect SQL injection with UNION SELECT in HTTP URI », avec l'architecture « Agentic RAG », `Top-k=5` et le modèle `mock` affiché par l'interface. Le modèle `mock` sert ici à obtenir une exécution locale déterministe du tableau de bord ; les champs de l'interface indiquent aussi des exemples de modèles locaux Ollama utilisables lorsque l'environnement les fournit.

10.1 Onglet Rule Generator

L'utilisateur saisit la description d'attaque, choisit l'architecture, règle le Top-k et déclenche « Generate rule ». Le système affiche la règle Snort candidate, le type d'attaque, le mode de génération, le modèle utilisé, l'état de validité locale, une explication et les fichiers d'export.

Rule Generator | Retrieval Details | Validation & False Positives | Dataset / Knowledge Base | Benchmarks & Evidence | PDF Upload

Generate defensive Snort rules from an attack description. *mock* is only for smoke testing; use examples such as `ollama:qwen2.5`, `ollama:mistral`, or `ollama:llama3.2` for local LLM runs.

Attack description: Detect XSS using script tag in a web request

Architecture: RAG + LLM controle

LLM model: ollama:qwen2.5

Top-k retrieval: 3 (range 2 to 10)

Generate rule

Generated Snort rule

```
1 alert tcp $EXTERNAL_NET any -> $HOME_NET 8080 (msg:"P1 XSS payload marker 6"; flow:to_server,established; content:"<iframe src='',nocase; pcre:/'<iframe src=javascript:alert(1)>/i"; classtype:web-application-attack; sid:9100066; rev:1;)
```

Attack type	Generation mode	Model name	Validity status
xss	template_fallback_after_llm_failure	qwen2.5	VALID

Deterministic fallback was used after controlled validation or because a non-LLM architecture was selected.

Explanation: Attack type: xss. The generated rule declares classtype (web-application-attack) and includes msg/sid/rev. It was selected or adapted after retrieving similar examples: SNORT-P1-0066, SNORT-P1-0074, SNORT-P1-0061. Parser status: valid by internal parser.

Download latest .rules export: latest_generated_rule.rules (237.0 B)

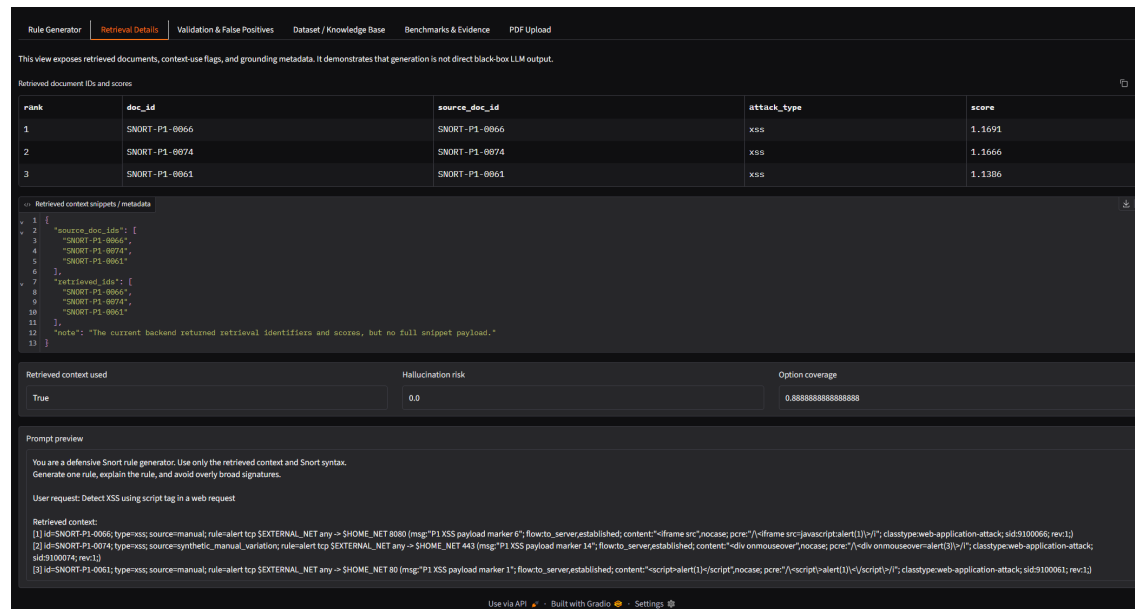
Download latest JSON export: latest_generation_result.json (5.2 KB)

FIGURE 10.1 : Onglet « Rule Generator » après génération d'une règle pour une injection SQL contenant UNION SELECT.

Source : capture réelle du tableau de bord Gradio du projet.

Le flux visible relie la saisie utilisateur à `templates.py`, `retrieval.py`, `architectures.py`, `generator.py` ou `llm_generator.py`, puis `rule_parser.py`. Pour cette requête, le tableau de bord détecte `sql_injection`, récupère des exemples proches et affiche une règle valide selon le parseur interne. Cette validation locale ne remplace pas la validation Snort 3 runtime ni le rejeu PCAP.

Cet onglet expose les documents récupérés pour la même requête. L'utilisateur inspecte les identifiants de documents, rangs, catégories d'attaque, scores, métadonnées de source, indicateur d'utilisation du contexte, risque d'hallucination, couverture d'options et aperçu du prompt.

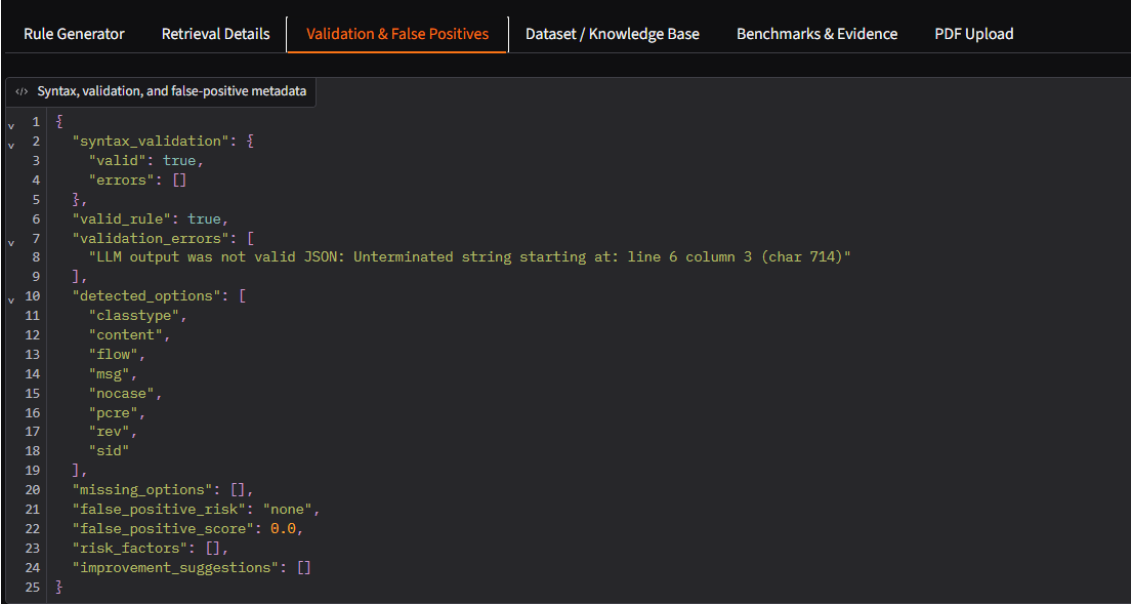


Source : capture réelle du tableau de bord Gradio du projet.

Le Top-k fixe le nombre de documents candidats visibles. Le module `retrieval.py` combine notamment TF-IDF et BM25 dans les variantes hybrides, puis certaines architectures appliquent un reclassement. Ces documents servent de contexte contrôlé pour rapprocher la réponse d'exemples Snort existants, sans garantir à eux seuls la correction opérationnelle d'une règle.

10.3 Onglet Validation & False Positives

Cet onglet présente le résultat de validation syntaxique locale, les erreurs éventuelles, les options Snort détectées, les options manquantes, la couverture d'options, le score de risque de faux positifs, les facteurs de risque et les suggestions d'amélioration.



```
1 {  
2   "syntax_validation": {  
3     "valid": true,  
4     "errors": []  
5   },  
6   "valid_rule": true,  
7   "validation_errors": [  
8     "LLM output was not valid JSON: Unterminated string starting at: line 6 column 3 (char 714)"  
9   ],  
10  "detected_options": [  
11    "classtype",  
12    "content",  
13    "flow",  
14    "msg",  
15    "nocase",  
16    "pcrc",  
17    "rev",  
18    "sid"  
19  ],  
20  "missing_options": [],  
21  "false_positive_risk": "none",  
22  "false_positive_score": 0.0,  
23  "risk_factors": [],  
24  "improvement_suggestions": []  
25 }
```

FIGURE 10.3 : Onglet « Validation & False Positives » affichant les métadonnées de validation et de risque.

Source : capture réelle du tableau de bord Gradio du projet.

La validation locale vient de `rule_parser.py`; elle vérifie la structure de la règle et les options détectables par le parseur interne. L'analyse de faux positifs vient de `false_positive.py`; elle signale les règles trop larges, les options absentes ou les situations à surveiller. Ces contrôles restent distincts d'une validation Snort 3 réelle et d'une validation comportementale PCAP.

10.4 Onglet Dataset / Knowledge Base

L'interface affiche la taille du dataset, les chemins de preuve, la distribution par catégorie, la distribution par type de source et un échantillon du corpus. Elle distingue le corpus officiel, les journaux synthétiques, les journaux de laboratoire et la base externe optionnelle.

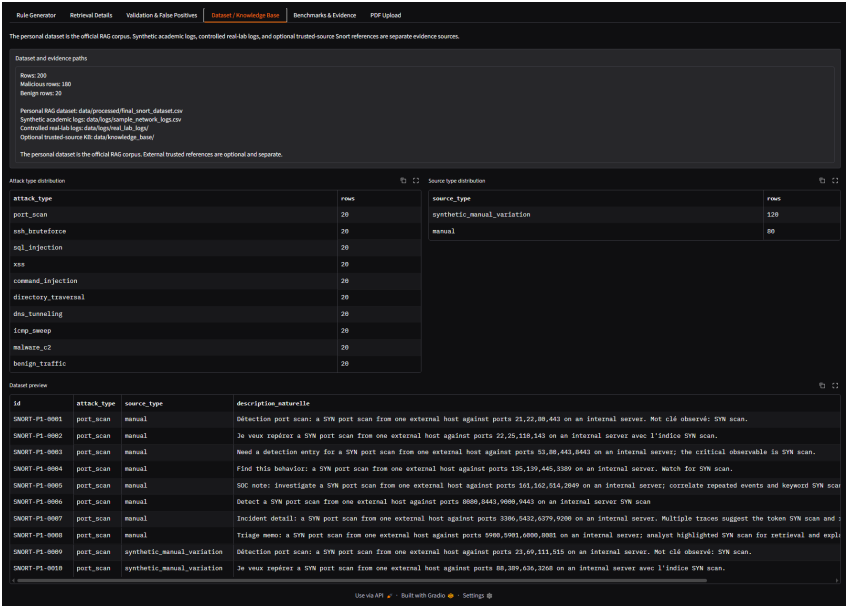


FIGURE 10.4 : Onglet « Dataset / Knowledge Base » résumant le corpus officiel et les sources de connaissance séparées.

Source : capture réelle du tableau de bord Gradio du projet.

Le corpus officiel contient 200 lignes réparties en dix catégories équilibrées : neuf catégories malveillantes et **benign_traffic**. Les exemples proviennent de lignes manuelles et de variations synthétiques contrôlées. Les exemples bénins utilisent **NO_RULE_RECOMMENDED**, car une règle d’alerte sur un trafic normal augmenterait les faux positifs.

10.5 Onglet Benchmarks & Evidence

Cet onglet regroupe les cartes de statut et les tableaux de preuves sauvegardées : benchmark LLM, benchmark d’embedding, clustering, résultats PCAP, validation runtime Snort et intégration de journaux de laboratoire.

LLM benchmark summary									
model_spec	prompt_variant	queries	ok_rows	error_rows	valid_rule_rate	attack_type_accuracy	retrieval_grounding_rate	avg_latency_ms	
ollama:llama3.2	strict_json	10	10	0	1	1	1	9730.3025	
ollama:mistral	strict_json	10	10	0	1	1	1	32517.1986	
ollama:qwen2.5	strict_json	10	10	0	1	1	1	22631.499600000003	

Embedding benchmark									
method	backend	k	status	sentence_bert_runtime_available	faiss_runtime_available	sentence_bert_model	faiss_enabled	error_message	hit_at_k
dense_tfidf	tfidf	5	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		1
dense_tfidf_rerank	tfidf_rerank	1	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		1
dense_tfidf_rerank	tfidf_rerank	3	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		1
dense_tfidf_rerank	tfidf_rerank	5	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		1
hybrid_rerank_best	hybrid_tfidf_bm25_rerank	1	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		1
hybrid_rerank_best	hybrid_tfidf_bm25_rerank	3	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		1
hybrid_rerank_best	hybrid_tfidf_bm25_rerank	5	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		1
hybrid_alpha_0.25	hybrid_tfidf_bm25	1	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		0.9
hybrid_alpha_0.25	hybrid_tfidf_bm25	3	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		0.9666666666666668
hybrid_alpha_0.25	hybrid_tfidf_bm25	5	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		0.9600000000000002
hybrid_alpha_0.50	hybrid_tfidf_bm25	1	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		0.9
hybrid_alpha_0.50	hybrid_tfidf_bm25	3	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		0.9666666666666668
hybrid_alpha_0.50	hybrid_tfidf_bm25	5	ready	true	true	sentence-transformers/all-MiniLM-L6-v2	false		0.9600000000000002

Clustering metrics							
method	n_clusters	silhouette	ari	nmi	homogeneity	completeness	v_measure
tfidf_svd_kmeans	10	0.4326666348323723	1	1	1	1	1
tfidf_svd_agglomerative	10	0.4326666348323723	1	1	1	1	1

FIGURE 10.5 : Onglet « Benchmarks & Evidence » affichant les preuves expérimentales déjà calculées.

Source : capture réelle du tableau de bord Gradio du projet.

Les données affichées proviennent notamment de `llm_benchmark_summary.csv`, `embedding_benchmark.csv`, `clustering_metrics.csv`, `pcap_test_results.csv`, `snort_runtime_validation.csv` et `real_lab_log_integration_eval.csv`. L’onglet visualise ces artefacts ; il ne relance pas tous les benchmarks à l’ouverture et ne transforme pas ces preuves en certification opérationnelle.

10.6 Onglet PDF Upload

L'utilisateur sélectionne un fichier PDF, clique sur « Index uploaded PDF », puis l'interface affiche le nom du fichier, le nombre de chunks ajoutés, la nouvelle taille de la base de connaissances et un aperçu du premier chunk importé.

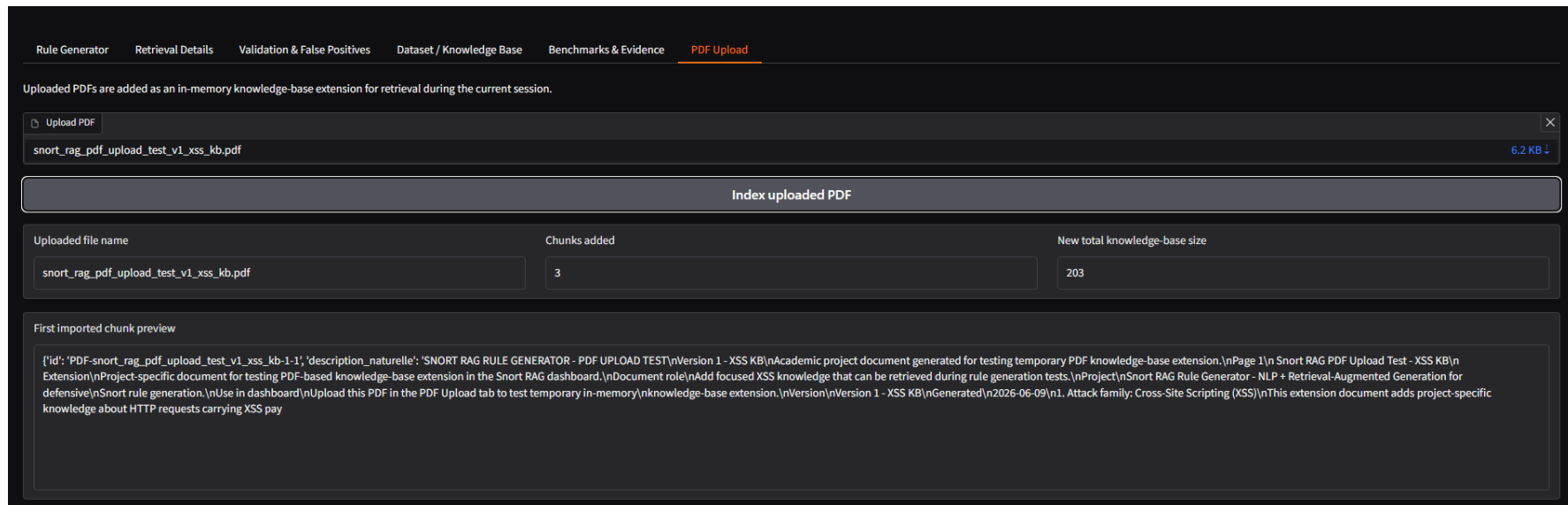


FIGURE 10.6 : Onglet « PDF Upload » après indexation temporaire d'un PDF dans la base de connaissances en mémoire.

Source : capture réelle du tableau de bord Gradio du projet.

Le traitement s'appuie sur la base de connaissances utilisée par `retrieval.py` : le PDF est lu, découpé en chunks, puis ajouté en mémoire pour la session courante. Cette extension est temporaire et ne remplace pas le dataset officiel. Le contenu téléversé reste non fiable tant qu'il n'est pas vérifié.

10.7 Démonstration bénigne contrôlée

Le cas contrôlé « Normal employee browsing the company website with regular HTTP traffic » vérifie que le tableau de bord peut refuser une génération de règle lorsque la description correspond à un trafic normal.

Rule Generator Retrieval Details Validation & False Positives Dataset / Knowledge Base Benchmarks & Evidence PDF Upload

Generate defensive Snort rules from an attack description. *work is only for smoke testing; use examples such as ollama:qwen2.5, ollama:mistral, or ollama:llama3.2 for local LLM runs.*

Attack description

Normal user browsing HTTPS and doing a standard DNS lookup without suspicious payloads

Architecture

RAG + LLM controle

LLM model

ollama:qwen2.5

Top-k retrieval

2 3 10

Generate rule

Generated Snort rule

1 NO_RULE_RECOMMENDED

Attack type	Generation mode	Model name	Validity status
benign_traffic	rag_llm	qwen2.5	VALID

Explanation

The user's activity is normal HTTPS browsing and standard DNS lookup, which does not warrant any defensive IDS rules.

Download latest_rules export

latest_generated_rule.rules 0.0 B

Download latest_JSON export

latest_generation_result.json 5.6 KB

Use via API Built with Gradio Settings

FIGURE 10.7 : Onglet « Rule Generator » retournant NO_RULE_RECOMMENDED pour un trafic HTTP normal.

Source : capture réelle du tableau de bord Gradio du projet.

Cette décision est correcte dans le cadre du système : une description de navigation normale ne doit pas forcer la création d'une signature IDS. Générer une règle pour ce trafic augmenterait le risque de faux positifs et diminuerait la valeur opérationnelle des alertes.

10.8 Synthèse de bout en bout

Le tableau 10.1 résume les étapes visibles de la démonstration et les modules principaux impliqués.

TABLEAU 10.1 : Synthèse fonctionnelle de la démonstration Gradio.

Étape	Action dans l'interface	Module principal	Résultat visible
Entrée	Description ou journal	<code>app_gradio.py</code>	Requête utilisateur
Détection	Analyse initiale	<code>templates.py</code>	Type d'attaque
Récupération	Top-k	<code>retrieval.py</code>	Documents et scores
Génération	RAG ou déterministe	<code>llm_generator.py</code> / <code>generator.py</code>	Règle ou décision bénigne
Validation	Analyse locale	<code>rule_parser.py</code>	Validité et options
Faux positifs	Analyse heuristique	<code>false_positive.py</code>	Risque et suggestions
Preuves	Consultation	fichiers <code>results/</code>	Benchmarks et validations
PDF	Extension temporaire	base de connaissances	Nouveaux chunks en mémoire

11 Conclusion générale

La problématique demandait dans quelle mesure un système NLP/RAG contrôlé peut produire des règles Snort 3 candidates, traçables et validables sans surestimer les garanties offertes. La réponse est nuancée. Le système produit des réponses structurées et validées localement, et il s'appuie sur une récupération performante dans un corpus contrôlé. Cependant, une partie importante des résultats LLM sauvegardés repose sur la solution de repli déterministe, et plusieurs validations comportementales sont hors ligne.

Les preuves recalculées corrigent trois points essentiels. Premièrement, le dataset officiel contient 180 SID exportés, tandis que le fichier Snort 3 validé contient 183 SID parce que trois règles supplémentaires servent aux PCAP. Deuxièmement, la récupération et le clustering doivent être interprétés avec une analyse de fuite d'étiquette. Troisièmement, un PCAP malveillant ne doit pas être considéré réussi parce qu'une alerte quelconque apparaît : le SID ou la famille attendue doit correspondre.

Le projet constitue donc une base académique cohérente pour étudier la génération contrôlée de règles Snort 3 par NLP/RAG. Il ne constitue pas une validation de production. Les travaux futurs doivent élargir les corpus et les PCAP, isoler les jeux de réglage et d'évaluation, instrumenter précisément les LLM locaux, et remplacer les heuristiques par des mesures comportementales plus robustes.

Bibliographie

- [1] Cisco Talos, *Snort 3 User Manual*, n.d. Consulté le 13 juin 2026.
- [2] K. Sparck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [3] S. Robertson and H. Zaragoza, “The probabilistic relevance framework : BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT : Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992, 2019.
- [5] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [7] Ollama, *Ollama Documentation*, n.d. Consulté le 13 juin 2026.
- [8] Hugging Face, *Gradio Documentation*, n.d. Consulté le 13 juin 2026.
- [9] FastAPI, *FastAPI Documentation*, n.d. Consulté le 13 juin 2026.